Test Project

[Web Technologies] - Module A (Module 1)

Speed Test Project

Modified and Developed Experts:
Thomas Galih Satria (ID) (Team Leader),
Mohammad(IR)
Huraizul(BN)
Wan Ahmad(MY)
Smita JhaJharia (AU)

Reviewed and Revised by Chief Expert (Chorng-Shiuh Koong)

Version 2.0

WorldSkills Asia acknowledges WorldSkills International's copyright and intellectual property rights. This document has been adapted for WorldSkills Asia with express permission from WorldSkills International.

Table of contents

Introduction 4

Tasks for Design Implementation 4

A1: It's Rainy (Level 1) 4

A2: Wave Text (Level 2) 4

A3: Barcode Scanner Animation (Level 2) 5

A4: Tunnel Animation (Level 1) 6

A5: Multi-Step Form (Level 3) 6

A6: Tooltip Chain (Level 3) 7

A7: Loading Animation (Level 1) 8

A8: Loading Animation (Level 2) 8

A9: Create a functional loading animation (Level 1) 9

A10: Drip (Level 2) 10

A11: CSS Bar Chart (Level 1) 10

Tasks for Front-end Development 11

B1: Infinite Scroll Quotes Viewer (Level 1) 11

B2: CSS Inspector (Level 2) 11

B3: 3D Infinite Tunnel (Level 2) 12

B4: Lightning Strike (Level 1) 13

B5: ASCII Real-Time Drawing (Level 3) 13

B6: Video Scrubber (Level 3) 14

B7: Magnetic Draggable Elements (Level 2) 15

B8: Dynamic Notebook (Level 2) 15

B9: Symbol Password Mask (Level 1) 15

B10: Word Split Shuffle (Level 1) 16

B11: Image Carousel (Level 2) 16

B12: Drawing Tool (Level 3) 17

B13: Line Chart (Level 2) 18

B14: Image Compare (Level 2) 19

B15: Fractal Triangle (Level 1) 20

B16: Multiple Counters (Level 1) 21
Tasks for Back-end Development 22
C1: Guestbook with Pagination (Level 1) 22
C2: CSV Data Parsing and Aggregation (Level 2) 22
C3: File Content Search (Level 2) 22
C4: Timeline Conflict Resolver (Level 3) 23
C5: Crossword Generator (Level 3) 23
C6: Anagram Finder (Level 1) 23
C7: Hierarchical Menu Builder (Level 1) 23
C8: ASCII Art Converter (Level 2) 24
C9: File-Based URL Shortener (Level 2) 25
C10: Folder Zip (Level 1) 25
C11: API CORS (Level 2) 25
C12: Image Mosaic (Level 1) 26
Instructions to the Competitor 27
Other 27
Marking Summary 27

Introduction

In this mini speed Test Project, you are given multiple mini tasks. We have collected all the tasks from multiple clients. And for priority reasons, we have selected some tasks for this round of development. Please only implement the required tasks.

Please follow the instructions to know which one to implement.

For each mini Test Project, there are three levels.

- **Level 1: 1.0 points worth that is expected takes less than 15 minutes of work.**
- **Level 2: 2.0 points worth that is expected takes less than 20 minutes of work.**
- **Level 3: 3.0 points worth that is expected takes around 30 minutes of work.**

Please put your files in /XX_module_a/ and with the task ID as folder name, e.g. /XX_module_a/A1/ The

XX is the workstation number.

Tasks for Design Implementation

A1: It's Rainy (Level 1)

Create a rain animation effect using only **HTML and CSS.**
Randomly generated raindrops should fall continuously from the top of the screen to the bottom.

Please refer to **demo.mp4**

Requirement Notes:

- The background should cover the entire browser window.
- Each raindrop is a thin vertical line.
- The height of each raindrop must be between **10px and 200px**.
- The horizontal position of each raindrop is distributed randomly across the screen.
- Raindrops move smoothly from the top of the screen to beyond the bottom edge.
- The rain should continue infinitely.

A2: Wave Text (Level 2)

Create a wave animation effect on text using only HTML and CSS.
The text should animate up and down sequentially, giving the illusion of a wave.

Please refer to **demo.mp4**

Requirement Notes:

- The text must be exactly: **“WorldSkills Asia Competition Taipei 2025”**.
- Each character should have a slight delay, so the movement looks like a flowing wave from left to right.
- Only HTML and CSS are allowed. No JavaScript.
- The text should be horizontally centered and vertically centered in the viewport.

A3: Barcode Scanner Animation (Level 2)

Create a barcode design with a moving red scanner line using only HTML and CSS.

Please refer to **demo.mp4**



Requirement Notes

- The barcode must consist of multiple vertical black and white bars with varying widths.
  Bars must be placed inside a bordered box, horizontally aligned.
- Below the barcode, display the number: **162956944404**.
- The barcode should be vertically and horizontally centered in the viewport.
- The design must use **only HTML and CSS**. JavaScript is not allowed.

A4: Tunnel Animation (Level 1)

Create a 3D tunnel animation effect using only HTML and CSS.
Multiple rings should appear to move continuously towards the viewer, giving a sense of traveling through a tunnel.

Please refer to **demo.mp4**

Requirement Notes

- The background must be black and cover the entire viewport.
- The opacity of rings should change smoothly as they move, fading in and out naturally.
- The tunnel effect should be centered both vertically and horizontally in the viewport.
- The animation should loop infinitely.
- Only HTML and CSS are allowed. JavaScript is not permitted.

A5: Multi-Step Form (Level 3)

Create a multi-step form using only HTML and CSS. The form should allow users to navigate between steps without JavaScript, relying purely on CSS.

Please refer to **demo.mp4**

Requirement Notes

- The form has **3 steps**:
    1. **Personal Information**: *Name*, *Email*
    2. **Address**: *Street*, *City*
    3. **Confirm**: confirmation text and *Submit* button
- Only one step is visible at a time.
- Step transitions must be smooth (e.g., fade or slide effect).
- The form must be horizontally and vertically centered in the viewport.

A6: Tooltip Chain (Level 3)

Create a tooltip chain interaction using only HTML and CSS. When the user hovers on the trigger element, a tooltip should appear. Hovering on that tooltip reveals another tooltip, and hovering on the second reveals a third, forming a **sequential chain** of tooltips.

Please refer to **demo.mp4**

Requirement Notes:

- Must use only **HTML and CSS** (no JavaScript allowed).
- There are exactly **3 tooltips** chained:
    - **First tooltip**: appears when hovering the trigger.
    - **Second tooltip**: appears when hovering the first tooltip.
    - **Third tooltip**: appears when hovering the second tooltip.
- Tooltips must have **arrow indicators** pointing to their parent element.
- Each tooltip should have a slight fade-in effect on hover.
- Only one level of tooltip should appear at a time unless hovered in sequence.

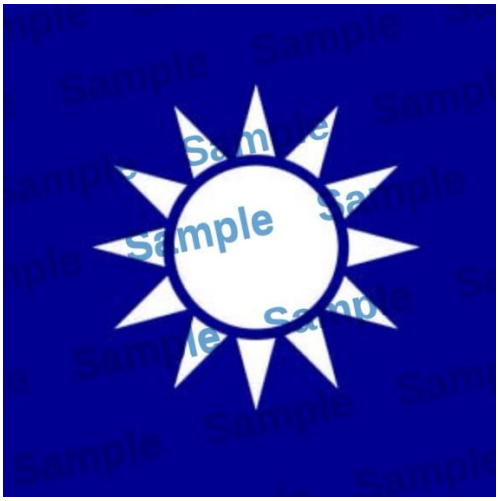
A7: Loading Animation (Level 1)

CSS - Implement a loading animation similar to the provided GIF



A8: Loading Animation (Level 2)

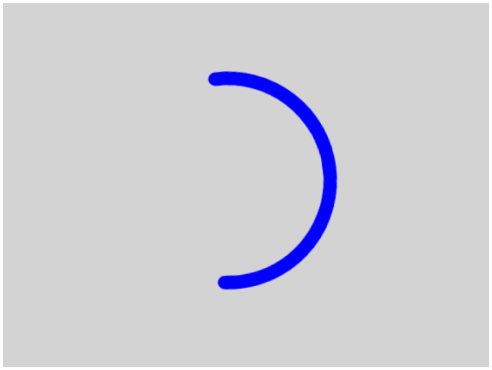
Implement the symbol using HTML and CSS.



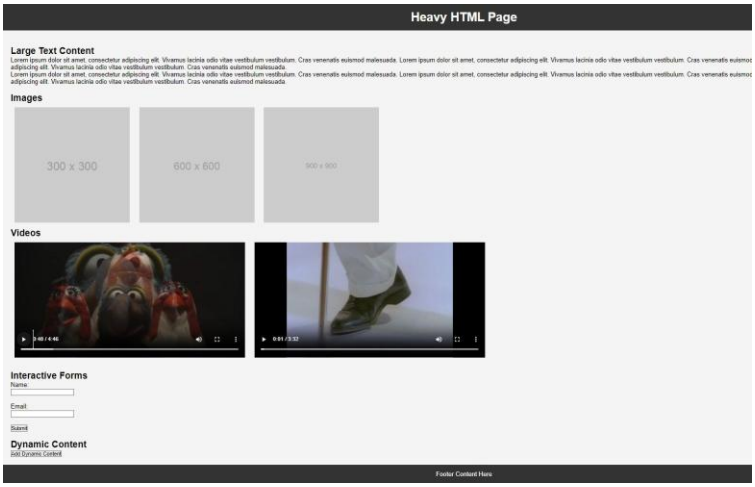
A9: Create a functional loading animation (Level 1)

Watch the video and recreate the spinner loading animation. The animation should only run while the provided HTML page is loading. The browser must be settled with a slow 3G connection and 6x slowdown CPU to test this.

Loading animation example:



Page displayed after animation:



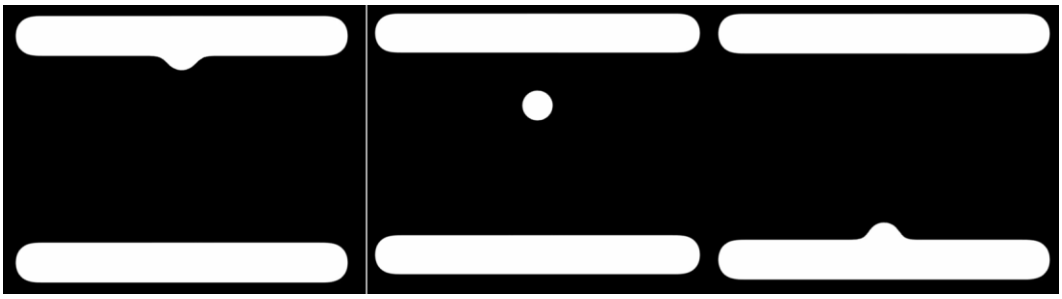
Requirement Notes

- The loading animation have been created
- The loading animation follows the provided media files
- The loading animation is displayed only while the page is loading

A10: Drip (Level 2)

No code is provided. Please implement a looping drip effect only using HTML and CSS, following the provided video.

These are some screenshots from the video.



Requirement Note

- Please refer to the reference video and create the same effect.

A11: CSS Bar Chart (Level 1)

Please implement a bar chart using only HTML and CSS.

At first, the screen will be blank, but when you hover the mouse over the chart area, the bars will rise sequentially from the bottom, starting from the left bar.

The values and colors of each data point are not important. You

can use html and css, but not javascript

Please refer to resource/demo.mp4

Requirement Notes

- The bar graph area is displayed and the initial screen is blank.
- When you place the mouse over the chart area, the bars move up in order from the bottom.
- When you place the mouse over the chart area, the bar goes up sequentially starting from the left

Tasks for Front-end Development

B1: Infinite Scroll Quotes Viewer (Level 1)

Create a continuously loading quote viewer using HTML, CSS, and JavaScript.

Please refer to **demo.mp4**

Requirement Notes:

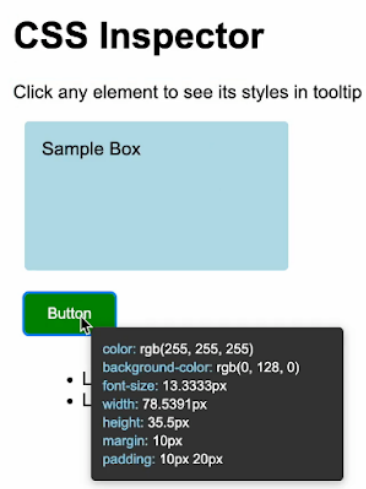
The infinite scroll viewer must:

- Display a list of quotes with both the text and the author clearly formatted.
- Automatically load more quotes as the user scrolls near the bottom of the page.
- Load quotes in batches (e.g., 10 per batch) and show a **"Loading more quotes…"** message while fetching new quotes.
- Show a **"No more quotes available"** message when all quotes have been loaded.
- Fetch all quotes from a **quotes.json file**, provided in the media files.

B2: CSS Inspector (Level 2)

Create an interactive CSS inspector tool using HTML, CSS, and JavaScript.

Please refer to **demo.mp4**



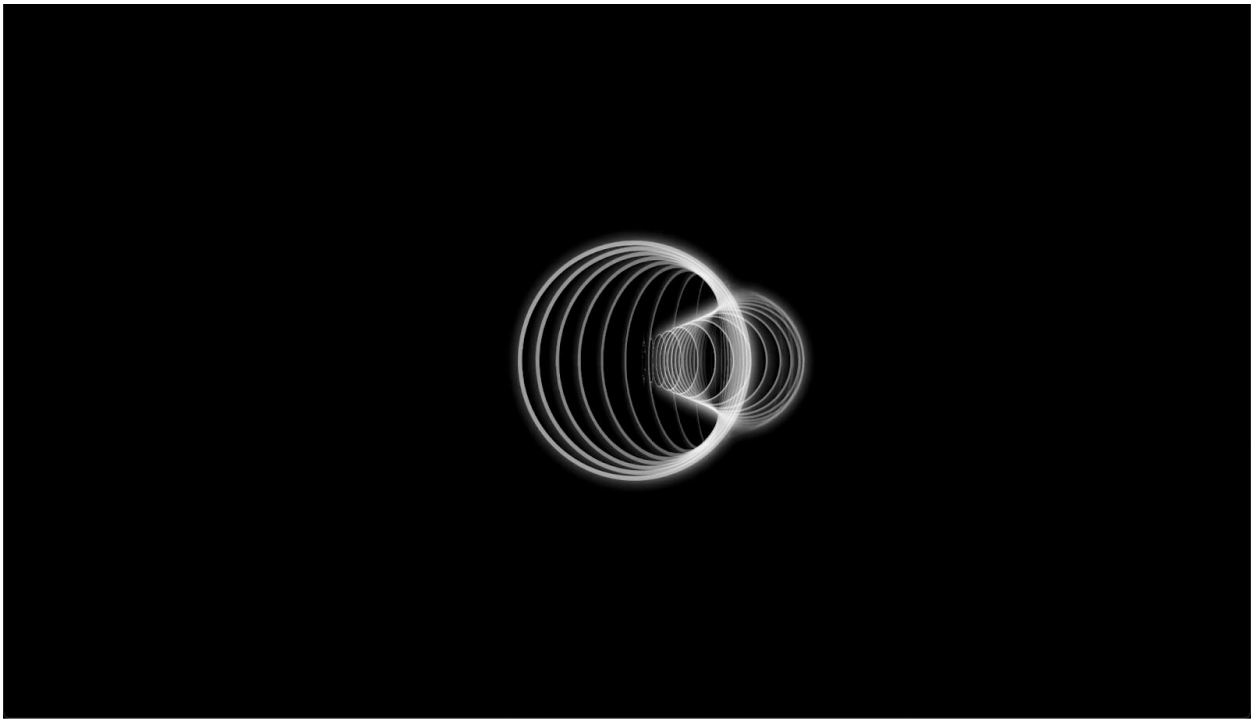
Requirement Notes

- The inspector must:
    - Highlight any element the user **hovers** over using a visible border or outline.
    - Display a **tooltip** showing selected CSS properties when the user clicks an element.
- Displayed CSS properties must include at least:
    - color, background-color, font-size, width, height, margin, padding.
- Highlighted elements should **remove highlights** when a new element is hovered.
- Sample elements (boxes, buttons, lists) must be provided for demonstration.
- Implementation must be done using **JavaScript only** (no external libraries).

B3: 3D Infinite Tunnel (Level 2)

Create an animated infinite tunnel effect using HTML, CSS, and JavaScript.

Please refer to **demo.mp4**



Requirement Notes:

- Rings must:
    - Have increasing diameters as they move outward.
    - Be circular with white semi-transparent borders.
    - Include subtle **glow effects** (box-shadow).
- The tunnel must appear **infinite**, with rings fading in and out while scaling.

B4: Lightning Strike (Level 1)

Create an interactive lightning effect, clicking anywhere on the screen triggers a lightning strike from the top of the screen down to the clicked X position.

Please refer to **demo.mp4**.

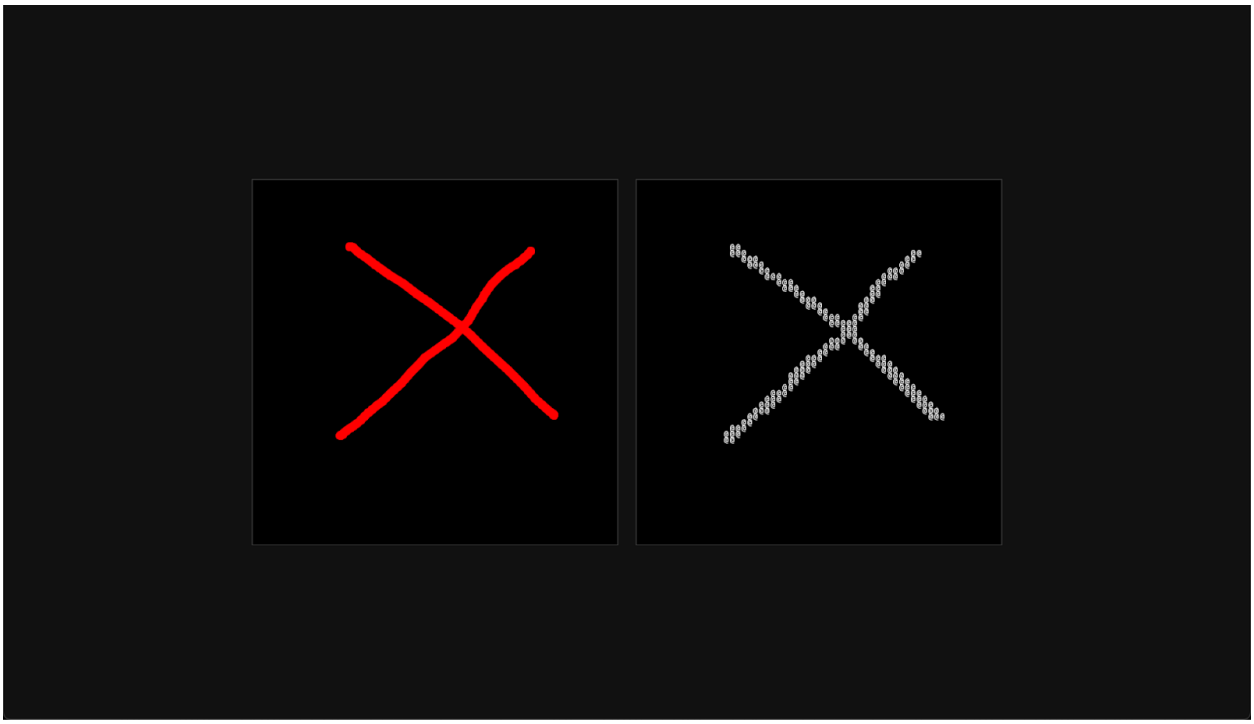
Requirement Notes:

- Each strike should consist of multiple jagged segments to simulate natural lightning.
- Multiple lightning bolts can appear simultaneously per click (randomized count).
- Lightning should fade or clear shortly after appearing.

B5: ASCII Real-Time Drawing (Level 3)

Build an interactive dual-canvas app where freehand drawing on one canvas is transformed into ASCII art in real time on the second canvas.

Please refer to **demo.mp4**.



Requirement Notes:

- Two canvases side by side:
    - **First canvas**: user draws with mouse or touch.

    - **Second canvas**: renders corresponding ASCII characters (@) representing the drawing.

- The ASCII view updates in real time as the user draws.
- The effect should progressively reveal characters, simulating a typed-out ASCII display.
- Only areas where the drawing exists are converted into @ symbols.
- Must be implemented using **HTML, CSS, and JavaScript only**.

B6: Video Scrubber (Level 3)

Create a custom video player with interactive scrubber and thumbnail preview using HTML, CSS, and JavaScript.

Please refer to **demo.mp4**.



Requirement Notes:

The video player must:

- Include a **play/pause button** and update its icon accordingly.
- Show a **progress bar** reflecting the current playback position.
- Allow **click-to-seek** on the progress bar to jump to a specific time.
- Show a **thumbnail preview** above the progress bar when hovering over it:
    - Thumbnail must correspond to the hovered time.
    - Display the corresponding time next to the thumbnail.
    - Thumbnail disappears when the mouse leaves the progress bar.

- Smoothly handle seeking and rendering thumbnails without lag.
- Use **HTML, CSS, and JavaScript only** (no external libraries).

B7: Magnetic Draggable Elements (Level 2)

Create an interactive UI with draggable elements that respond to the mouse pointer with a magnetic attraction effect.

Please refer to **demo.mp4**.

Requirement Notes

- Include multiple draggable elements with distinct colors and labels.
- Elements must be movable using click-and-drag.
- Non-dragged elements should **magnetically react** when the mouse is nearby.
- Provide **visual feedback** on hover (e.g., subtle shadow or scaling).
- Elements should smoothly return to their original position when the pointer is away.

B8: Dynamic Notebook (Level 2)

Create a dynamic, multi-entry notebook interface where users can write notes in separate text areas.

Please refer to **demo.mp4**.

Requirement Notes

- The notebook should start with a single text area.
- Users can type freely; each text area **auto-resizes** to fit its content.
- Pressing **Enter** creates a new text area below the current one.
- **Shift + Enter** allows a new line within the same text area.

Create a password input field that replaces typed characters with random symbols while retaining the real value internally.

B9: Symbol Password Mask (Level 1)

Please refer to **demo.mp4**.

Requirement Notes

- The input must allow typing normally, but all visible characters should appear as randomly assigned symbols.

- The **real password value** can be revealed when the user clicks a toggle button.
- Pressing **Backspace** must remove both the real character and its corresponding symbol correctly, maintaining the correct cursor position.
- Pressing any regular key inserts the character at the cursor position and displays a new random symbol at that position.

B10: Word Split Shuffle (Level 1)

Create an interactive tool that splits a typed word into individual letters and allows users to shuffle them randomly.

Please refer to **demo.mp4**.

Requirement Notes

- Users can type a word into an input field.
- Each letter of the word should be displayed as an individual box below the input.
- Provide a **shuffle button** that randomly rearranges the letter boxes.
- The letter boxes should maintain smooth transitions and retain their styling after shuffling.

B11: Image Carousel (Level 2)

Create an image carousel that rotates while dragging or scrolling, with inertia looking effect similar to the provided video.
Scrolling down or drag to left makes ring rotates clockwise, otherwise, counterclockwise.

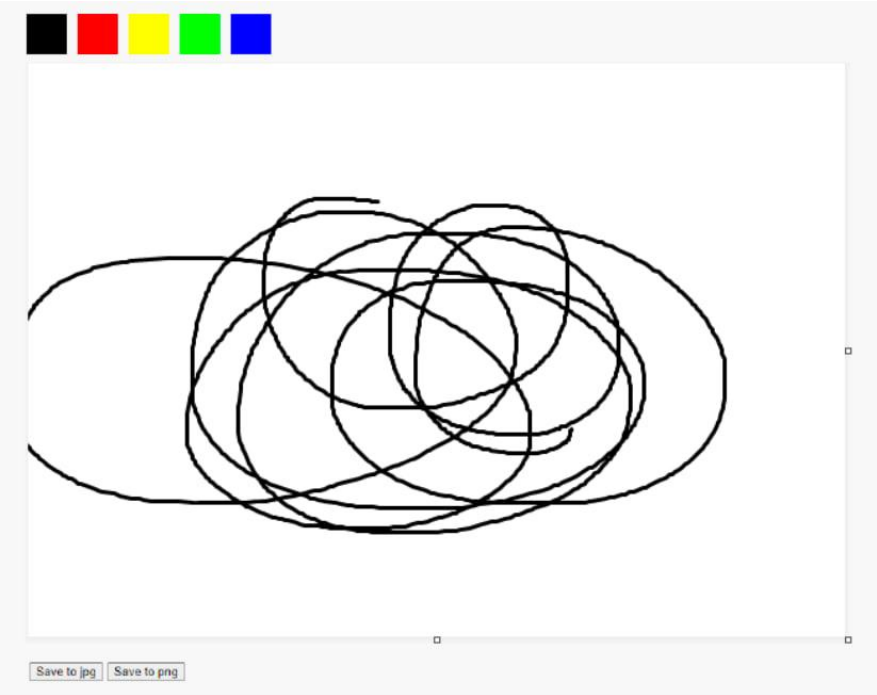


Please refer to **preview.mov**.

B12: Drawing Tool (Level 3)

You are going to create a client-side drawing tool, with a canvas on which one may freely draw.

- Provide five different color buttons for the user to choose from.
- The default canvas size must be 300px by 300px
- Clicking and dragging the bottom-right corner of the canvas should allow the user to resize the canvas.
- Provide one button to save the drawing as a JPG and another button to save it as a PNG on the client's computer.



B13: Line Chart (Level 3)

Create a line chart with visual effect for displaying website visits over the week.

Requirement Notes

- The chart should appear **blurry by default**.
- When the user **hovers the mouse over the chart**, the **blur effect should be removed**, making the chart clear.
- Show tooltip when mouse hover dot on the chart.
- Use **CSS** to implement the blur and hover effects.
- Data chart is a random data. Must be randomize when page is refresh.



B14: Image Compare (Level 2)

Given the HTML:

```html
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <title>Image Compare</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div>
        <input type="text">
        <img src="">
        <img src="">
    </div>
</body>
</html>
```

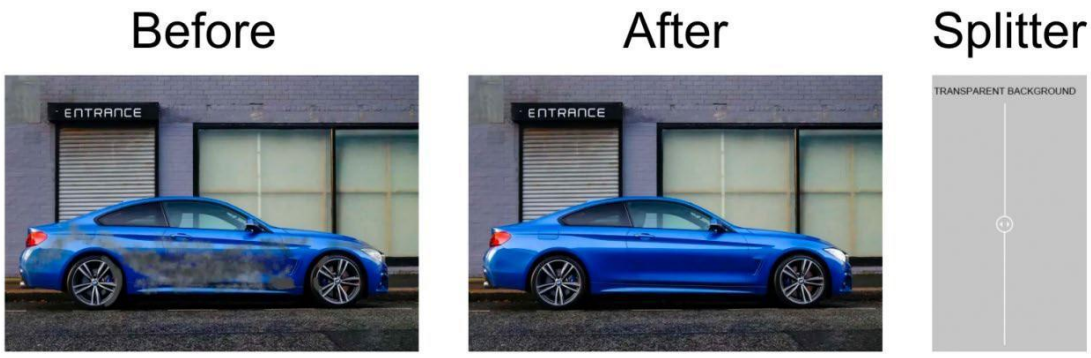
You need to use the provided index.html, style.css (empty file), two images (before.jpg and after.jpg) and a splitter.svg file to implement the Image Compare functionality. The Image Compare container has a splitter element. This element displays before.jpg on the left and after.jpg on the right. You can adjust the display ratio of the two images by clicking on the container or dragging the splitter element. As you adjust the ratio, one image will be displayed more prominently, while the other will be displayed less.

You cannot add or remove any HTML elements (except pseudo-elements). You can modify the attributes of existing HTML elements (including event listener attributes).

A sample video, "compare.mp4", is provided for reference. Below are some frames from the video:



The media files sample

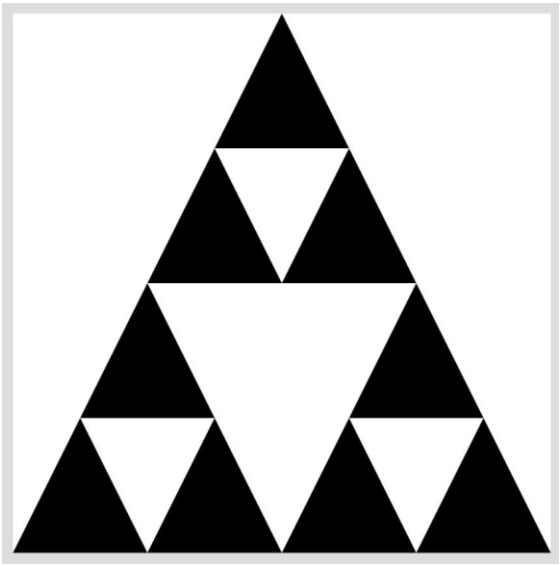


Requirement Notes

- The splitter left is before.jpg, the splitter right is after.jpg. No HTML elements were added or removed.
- The splitter can be moved by dragging with the mouse or clicking within the container.
- The image's effect is same as the compare.mp4 video.

B15: Fractal Triangle (Level 1)

After entering the number of iterations and pressing the button, you should draw the Fractal structure by repeating it the specified number of times.



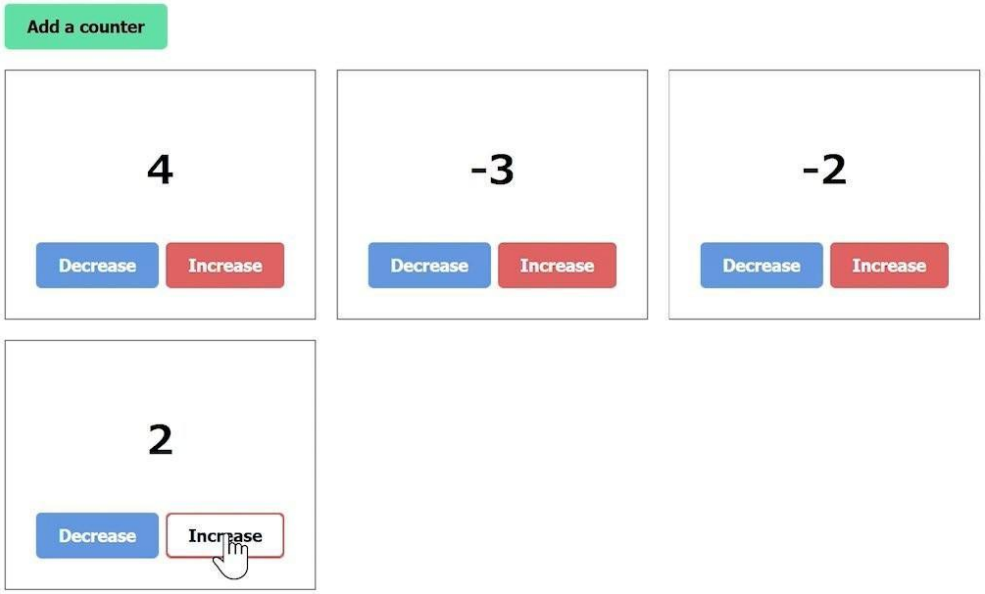
(Refer to demo.mp4)

Requirement Notes

- There is a canvas area, a form to input numbers, and a button.
- After entering the number of iterations and clicking the button, a fractal image is drawn.
- The drawn image has the same fractal structure as shown in the video (equilateral triangle structure).

B16: Multiple Counters (Level 1)

Using the provided video as a reference, complete the function using JavaScript.



Requirement Notes

- Can add a counter
- Can add multiple counters
- Each counter can independently increase and decrease
- UI is adjusted with user experience in mind

Tasks for Back-end Development

C1: Guestbook with Pagination (Level 1)

Create a simple guestbook. Users can submit their name and message, and messages are stored in a server-side file. Display the messages in **reverse chronological order** with pagination.

Requirement Notes:

- Save each entry as JSON in a text file (data.txt).
- Display messages newest first.
- Paginate messages, showing a limited number per page (configurable, e.g., 5).
- Include button for navigating pages (Prev, Next, page numbers).
- **No JavaScript allowed**.

C2: CSV Data Parsing and Aggregation (Level 2)

Create a PHP script that reads data from a predefined CSV string (you can define the CSV data directly in the script). The CSV data should contain at least two columns, one representing a category and the other representing a numerical value. The script should parse this data and calculate the sum of the numerical values for each unique category. The results should be outputted in a readable format (e.g., category: sum).

Requirement Notes:

- A predefined CSV string with at least two columns (category and numerical value) is used.
- The PHP script correctly parses the CSV data.
- The script calculates the sum of numerical values for each unique category.
- The aggregated results are outputted in a clear format.

C3: File Content Search (Level 2)

Write a PHP script that allows users to enter a search keyword via a form and scans all .txt files in a predefined source/ folder.

The script reads each file line by line and displays lines containing the keyword (case-insensitive), along with filenames and line numbers.

You can refer to demo.mp4 to see how the feature works.

Requirement Notes:

- The search keyword is entered through an HTML form.

- The script scans all .txt files inside the source/ directory.
- It performs a case-insensitive line-by-line search in each file.
- Matching lines are displayed, grouped by filename, and include line numbers.
- If no matches or files are found, a message is displayed accordingly.

C4: Timeline Conflict Resolver (Level 3)

Create a PHP script that accepts a JSON array of events, each with a title, start time, end time, and priority. Resolves overlaps by greedily keeping higher-priority events and removing conflicts. The final output must be a conflict-free schedule sorted by start time, along with a log of removed events based on priority. Input is submitted via a form, processed server-side, and displayed with the final JSON result. For clarity, refer to demo.mp4 to see how the script should work.

C5: Crossword Generator (Level 3)

Create a crossword grid generator. Users can input a list of words, and the program will generate a **grid placing the words horizontally or vertically**, filling empty spaces with blank cells.

Requirement Notes

- Input: list of words (comma-separated or newline).
- Display a **square grid** with size 15 x 15.
- Words are placed **randomly horizontally or vertically**.
- Overlapping letters must match correctly.
- Empty cells displayed as blank spaces.

C6: Anagram Finder (Level 1)

Create a web tool that finds **all anagrams** of a given word from a provided dictionary file.

Requirement Notes

- The application searches a **dictionary file (dictionary.txt)** for all valid anagrams of the input.
- The input word itself should **not** appear in the results.
- Display results in a clean, readable list on the page.
- Display a message if **no anagrams are found**.

C7: Hierarchical Menu Builder (Level 1)

Create a web tool that converts a plain-text category list into a **nested menu structure**.

Look at **demo.mp4** for example.

Requirement Notes:

- Users input categories line by line.
- Each line may represent a nested hierarchy using a **delimiter** (e.g., >).
- The application should parse the input and generate a **nested unordered list (<ul> /)** representing the hierarchy.
- Nested categories should appear as **submenus**.
- Display a preview of the menu below the form after submission.

C8: ASCII Art Converter (Level 2)

Create a web page that converts an image into ASCII art using only @ and . characters.



Requirement Notes:

- The image is scaled down to fit within a maximum **width of 200 characters** and **height of 80 characters**.

C9: File-Based URL Shortener (Level 2)

Create a simple web page that acts as a **URL shortener** using only PHP and a text file.

Requirement Notes:

- Users can input a full URL.
- A **short code** (6 characters) is generated and stored in a text file (urls.txt) along with the original URL.
- Visiting the short URL (?id=CODE) redirects to the original URL.
- Display an error if the short code does not exist or the input URL is invalid.

C10: Folder Zip (Level 1)

Please implement a simple web page for compressing folders. Users can select a folder (not empty) and click the "Compress" button to compress it into a zip file (The file name is the same as the uploaded folder name) and automatically download it. If the uploaded folder contains empty subfolders, those subfolders will not be included in the compressed archive. The folder for testing will be provided as media files

C11: API CORS (Level 2)

A Front-end application and a Back-end application are provided.

Access the Back-end through the VM Apache server, and use `npm run dev` for the Front-end application, making sure to use a different port than the Back-end.

When accessing the Front-end, 4 methods do not execute properly due to CORS.

Modify only the `cors.php` file in the Back-end application to ensure all requests execute correctly.

Run Front-end Application

You can run the development server by executing the `npm run dev` command.

Request address settings

You must open the front-end's `index.html` and modify the `API_HOST` variable to match your back-end. (node_modules are provided in a compressed file. Be sure to refer to README.md)

Requirement Notes

- When you open the development server and connect, requests are sent to the back-end application.
- The GET method request works correctly.
- The POST method request works correctly.
- The PUT method request works correctly.
- The DELETE method request works correctly.

C12: Image Mosaic (Level 1)

You need to dynamically create a mosaic of the provided image.

The completed image should be made up of square cells, with each cell having the average color value of the corresponding area of the original image.

The size of the cells can be set using the query parameter `cell_size`. If `cell_size` is not provided, it should be set to 50px.

Example) When cell size is 50px:



Requirement Notes

- The mosaic is created according to the size specified by the `cell_size` query parameter.
- If the `cell_size` query parameter is not provided, the mosaic is created with cells of size 50px.
- The mosaic image is the same size as the original image.

Instructions to the Competitor

Each mini Test Project folder should be separated and self-contained.

Please include only exactly the required mini test project. And do not include any not related folders or not related mini Test Projects.

Other

This project will be assessed by using Google Chrome web browser.

Marking Summary

	Sub-Criteria	**Marks**
1	Work organization and management	**0**
2	Communication and interpersonal skills	**0**
3	Design implementation	**8**
4	Front-end development	**8**
5	Back-end development	**8**
	Total Mark	**24**